

K L Deemed to be University

Name: Tej Pratap

Roll No: 2520030476

DATA STRUCTURES AND ALGORITHMS - 2

CO1 to CO3 Alternate Case Study Question Bank with Solution Keys

QUESTION PAPER STYLE CASE STUDIES

BST/AVL Trees • Segment Trees/Fenwick Trees • Graph Traversals & MST

Course	Data Structures and Algorithms - 2	Mode	Project-Based Learning
Coverage	CO1, CO2, CO3	BTL Level	4 - Apply/Analyze
Total Case Studies	3 complete alternate case studies	Marks Pattern	5 + 7 + 3 = 15 Marks each

This document contains fresh real-world case studies using the same CO topics as the earlier document, but with changed scenarios, input data, diagrams, questions, and solution keys.

Course Outcome Mapping

CO No	Selected Topic	Case Study Theme	BTL
CO1	Binary Search Trees and AVL Trees	E-commerce product-ID index with AVL rebalancing	4
CO2	Segment Trees and Fenwick Trees	Smart energy meter range-query system	4
CO3	Minimum Spanning Tree using Kruskal/Prim	IoT gateway network design under cable-cost limits	4

CASE STUDY 1 - CO1: BST and AVL Tree Indexing

CO	CO1	Topic	BST and AVL Trees
Scenario	E-commerce product catalogue search index	Marks	15
BTL	4	Main Skill	Construct, rebalance, and analyze AVL operations

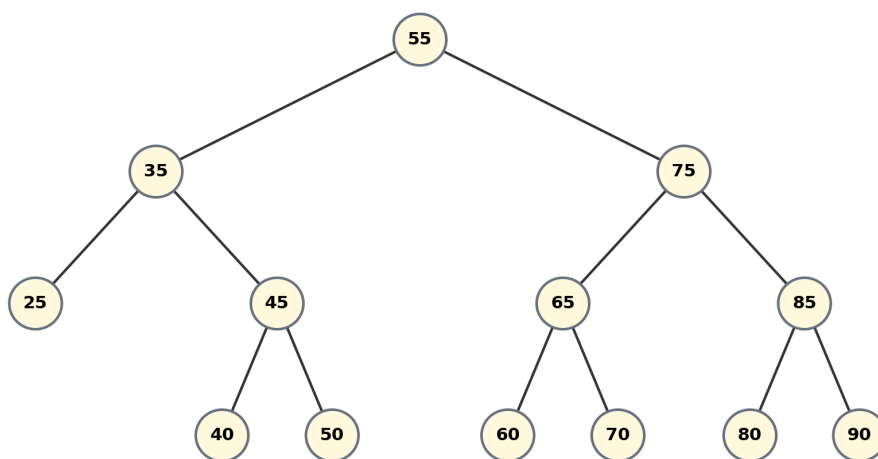
SCENARIO / CASE STUDY

ShopQuick, an e-commerce platform, keeps recently added product IDs inside an in-memory search index. The product manager wants product lookup to stay fast because customers search by product-ID during flash sales.

The product IDs arrive in this order: 55, 35, 75, 25, 45, 65, 85, 40, 50, 60, 70, 80, 90. A plain BST can become unsafe if future IDs arrive nearly sorted, so the backend team plans to use AVL balancing.

- Each pointer hop is estimated as 180 ns.
- The search SLA is $p99 < 4$ ms.
- AVL balance factor must stay in $\{-1, 0, +1\}$ after insertions and deletions.

Balanced AVL Index after Insertions



Sub	Question	Marks	CO/BTL
(a)	Construct the plain BST from the given product-ID insertion sequence. State the final height in edges and estimate worst-case lookup time using the 180 ns per hop model.	5	CO1/BTL4

Sub	Question	Marks	CO/BTL
(b)	Build the AVL tree from the same insertion sequence. Identify the insertions that trigger rebalancing and complete Java-style pseudocode for insert, rotateLeft, and rotateRight.	7	CO1/BTL4
(c)	If products 25, 75, and 55 are removed after the sale, explain the expected deletion impact on balance and state why AVL still guarantees $O(\log n)$ lookup.	3	CO1/BTL4

Solution Key - Case Study 1

Answer (a) - Plain BST construction and SLA check

Insertion behavior: The sequence begins around 55 and expands almost symmetrically, so this particular BST remains fairly balanced. Later values refine the left and right subtrees instead of forming a chain.

Height: The longest root-to-leaf path has 4 edges, for example 55 -> 35 -> 45 -> 40 or 55 -> 75 -> 85 -> 90 depending on representation.

Worst-case lookup: A search may examine 5 nodes. $5 \times 180 \text{ ns} = 900 \text{ ns} = 0.0009 \text{ ms}$. This is below the 4 ms SLA for this small set.

Safety: A plain BST is not structurally guaranteed. If product IDs later arrive in sorted order, height can become $n-1$, so AVL is safer for production.

Answer (b) - AVL rotations and code

Rotation trace: In this dataset, most insertions keep the tree balanced. If a future insertion creates LL, RR, LR, or RL imbalance, the first unbalanced ancestor is repaired immediately. The final AVL is height-balanced with 55 as the root in the shown layout.

Final AVL height: The final height is 3 edges, so lookup remains logarithmic.

```
class AVLNode {
    int key, height;
    AVLNode left, right;
}

int height(AVLNode n) { return n == null ? 0 : n.height; }
int balance(AVLNode n) { return n == null ? 0 : height(n.left) - height(n.right); }
void updateHeight(AVLNode n) { n.height = 1 + Math.max(height(n.left), height(n.right)); }

AVLNode rotateRight(AVLNode y) {
    AVLNode x = y.left;
    AVLNode T2 = x.right;
    x.right = y;
    y.left = T2;
    updateHeight(y); updateHeight(x);
    return x;
}

AVLNode rotateLeft(AVLNode x) {
    AVLNode y = x.right;
    AVLNode T2 = y.left;
    y.left = x;
    x.right = T2;
    updateHeight(x); updateHeight(y);
    return y;
}

AVLNode insert(AVLNode node, int key) {
    if (node == null) return new AVLNode(key);
    if (key < node.key) node.left = insert(node.left, key);
    else if (key > node.key) node.right = insert(node.right, key);
    else return node;
    updateHeight(node);
    int bf = balance(node);
    if (bf > 1 && key < node.left.key) return rotateRight(node);
    if (bf < -1 && key > node.right.key) return rotateLeft(node);
    if (bf > 1 && key > node.left.key) { node.left = rotateLeft(node.left); return rotateRight(node); }
    if (bf < -1 && key < node.right.key) { node.right = rotateRight(node.right); return rotateLeft(node); }
    return node;
}
```

Answer (c) - Deletion effect

Delete 25: This is a leaf/simple-child case, so only ancestors on the path to the root need height checks.

Delete 75: This is an internal node. Replace it using predecessor or successor, then rebalance upward.

Delete 55: Removing the root causes the largest structural change because the root must be replaced and both major subtrees must be height-checked.

Guarantee: After each deletion, AVL rotations restore balance factor to -1, 0, or +1, keeping search, insert, and delete operations $O(\log n)$.

CASE STUDY 2 - CO2: Segment Tree and Fenwick Tree Range Queries

CO	CO2	Topic	Segment Trees and Fenwick Trees
Scenario	Smart energy meter dashboard	Marks	15
BTL	4	Main Skill	Build, query, update, and compare range structures

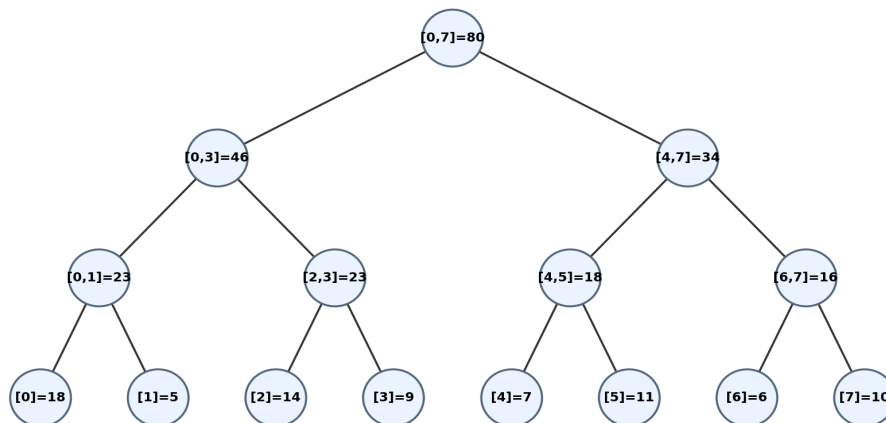
SCENARIO / CASE STUDY

A smart hostel dashboard records hourly electricity usage from 8 floors. The warden needs fast range-sum answers for selected floor groups and immediate updates when a meter reading is corrected.

Initial usage array by floor index 0 to 7 is: [18, 5, 14, 9, 7, 11, 6, 10]. The system receives $Q1 = \text{sum}(1,5)$, $Q2 = \text{sum}(0,7)$, then a point update index 2 from 14 to 20, followed by $Q3 = \text{sum}(1,5)$.

- Naive range sum scans every selected element.
- The dashboard needs $O(\log n)$ query and update performance.
- Use 0-based indexing and a sum Segment Tree.

Segment Tree for Daily Power Usage Array



Sub	Question	Marks	CO/BTL
(a)	Construct the Segment Tree for the given energy array. Write the root sum and internal sums for [0,3], [4,7], [0,1], [2,3], [4,5], and [6,7].	5	CO2/BTL4
(b)	Trace $Q1 = \text{sum}(1,5)$, $Q2 = \text{sum}(0,7)$, update index 2 from 14 to 20, and $Q3 = \text{sum}(1,5)$. Then write pseudocode for query and point update.	7	CO2/BTL4
(c)	Compare Segment Tree and Fenwick Tree for this energy dashboard. Which one is better if the system later needs range maximum query and why?	3	CO2/BTL4

Solution Key - Case Study 2

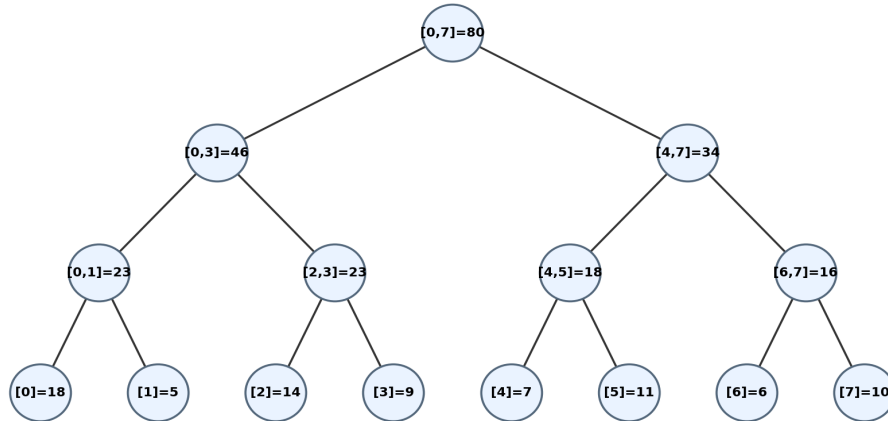
Answer (a) - Segment tree construction

Root: Total sum $[0,7] = 18 + 5 + 14 + 9 + 7 + 11 + 6 + 10 = 80$.

Internal sums: $[0,3] = 46$, $[4,7] = 34$, $[0,1] = 23$, $[2,3] = 23$, $[4,5] = 18$, $[6,7] = 16$.

Leaves: 18, 5, 14, 9, 7, 11, 6, 10. For 8 values, tree height is $\log_2(8) = 3$ edges.

Segment Tree for Daily Power Usage Array



Answer (b) - Query and update trace with code

Q1 sum(1,5): Use $[1,1] = 5$, $[2,3] = 23$, and $[4,5] = 18$. Total = 46.

Q2 sum(0,7): The root $[0,7]$ is a total-overlap node, so answer = 80.

Update index 2: Change 14 to 20. Recompute $[2,3] = 20 + 9 = 29$, $[0,3] = 23 + 29 = 52$, root $[0,7] = 52 + 34 = 86$.

Q3 sum(1,5): Use $[1,1] = 5$, $[2,3] = 29$, and $[4,5] = 18$. Total = 52.

```

int query(int node, int L, int R, int ql, int qr) {
    if (qr < L || R < ql) return 0;
    if (ql <= L && R <= qr) return tree[node];
    int mid = (L + R) / 2;
    return query(2*node, L, mid, ql, qr)
    + query(2*node+1, mid+1, R, ql, qr);
}

void update(int node, int L, int R, int idx, int val) {
    if (L == R) { tree[node] = val; return; }
    int mid = (L + R) / 2;
    if (idx <= mid) update(2*node, L, mid, idx, val);
    else update(2*node+1, mid+1, R, idx, val);
    tree[node] = tree[2*node] + tree[2*node+1];
}
  
```

Answer (c) - Segment Tree vs Fenwick Tree

Fenwick Tree: Very efficient and compact for prefix sums, range sums, and point updates. It is easier to implement when the operation has a prefix/inverse property.

Segment Tree: More flexible because each node can store sum, min, max, gcd, or custom merge information.

Choice: If future requirement is range maximum query, Segment Tree is better because maximum can be merged naturally from child nodes, while Fenwick Tree is less suitable for arbitrary range maximum with updates.

CASE STUDY 3 - CO3: Minimum Spanning Tree for IoT Gateway Design

CO	CO3	Topic	MST using Kruskal and Prim
Scenario	IoT gateway cable network	Marks	15
BTL	4	Main Skill	Apply MST algorithms and justify edge selection

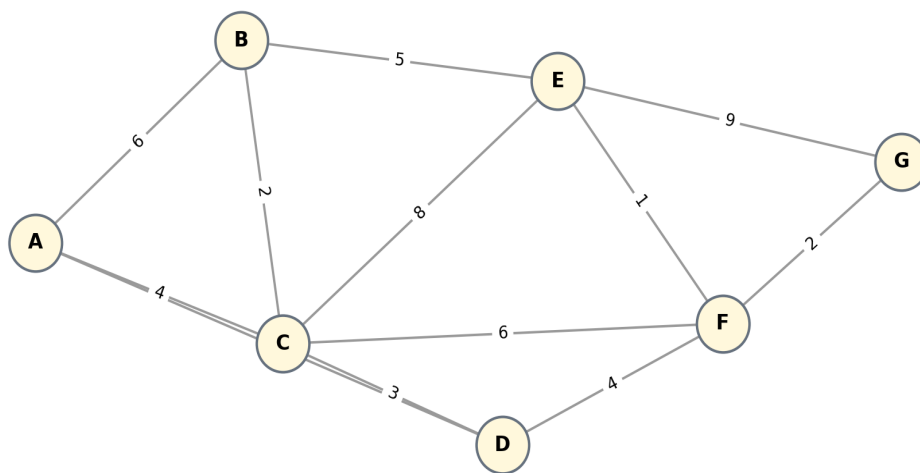
SCENARIO / CASE STUDY

A smart campus team wants to connect seven IoT gateway rooms using wired backbone links. Each link cost represents cable length and installation difficulty. The design must connect all gateway rooms with minimum cost and no cycles.

Gateway rooms are A, B, C, D, E, F, and G. Available routes are: AB=6, AC=4, AD=7, BC=2, BE=5, CD=3, CE=8, CF=6, DF=4, EF=1, EG=9, FG=2.

- Every gateway must be connected.
- The final design must avoid redundant cycles.
- Use Kruskal or Prim to construct the MST.

IoT Gateway Network Graph



Sub	Question	Marks	CO/BTL
(a)	Represent the weighted graph and sort all routes in non-decreasing order of cost. Explain why a spanning tree for 7 gateway rooms must contain exactly 6 edges.	5	CO3/BTL4
(b)	Apply Kruskal's algorithm step by step. Mention accepted/rejected edges and write Union-Find style pseudocode for cycle checking.	7	CO3/BTL4
(c)	If route EF becomes unavailable, identify a valid MST adjustment and explain the change in total cost.	3	CO3/BTL4

Solution Key - Case Study 3

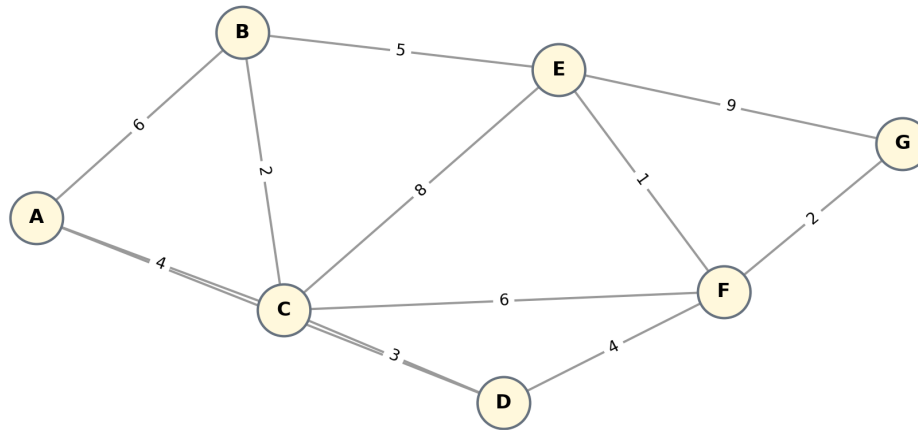
Answer (a) - Graph representation and edge order

Sorted edges: EF(1), BC(2), FG(2), CD(3), AC(4), DF(4), BE(5), AB(6), CF(6), AD(7), CE(8), EG(9).

Spanning tree size: A tree with n vertices contains $n - 1$ edges. Here $n = 7$, so the MST must contain 6 edges.

Reason: Fewer than 6 edges cannot connect all 7 rooms; more than 6 edges creates at least one cycle.

IoT Gateway Network Graph



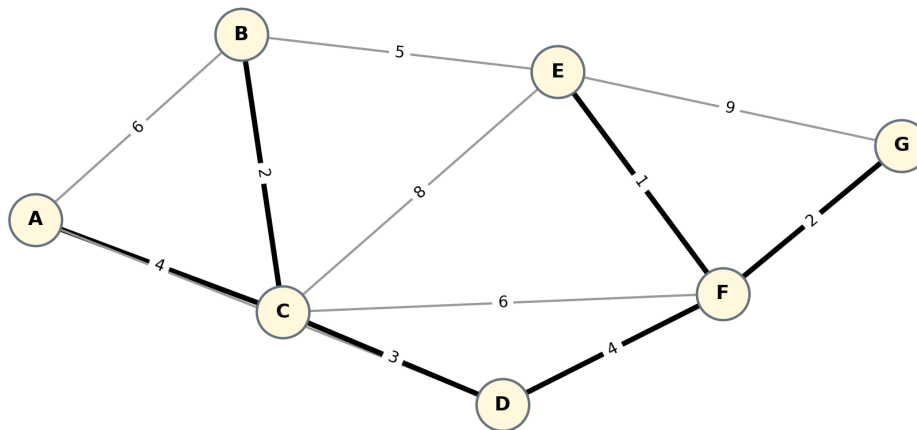
Answer (b) - Kruskal MST trace and code

EF(1): Accept. Connects E and F. BC(2): Accept. Connects B and C. FG(2): Accept. Connects G to E-F. CD(3): Accept. Connects D to B-C.

AC(4): Accept. Connects A to the B-C-D component. DF(4): Accept. Connects the A-B-C-D component to E-F-G. Now all 7 vertices are connected with 6 edges.

Final MST: {EF, BC, FG, CD, AC, DF}. Total cost = $1 + 2 + 2 + 3 + 4 + 4 = 16$.

IoT Gateway Network Graph - MST Highlighted



```
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false; // adding edge creates cycle
    if (rank[ra] < rank[rb]) parent[ra] = rb;
    else if (rank[ra] > rank[rb]) parent[rb] = ra;
    else { parent[rb] = ra; rank[ra]++; }
    return true;
}

for (Edge e : sortedEdges) {
    if (union(e.u, e.v)) addToMST(e);
    if (mstEdges == vertices - 1) break;
}
```

Answer (c) - Route failure adjustment

Unavailable edge: EF(1) is removed. The existing MST loses the cheapest E-F connection.

Adjustment: One valid new MST is {BC(2), FG(2), CD(3), AC(4), DF(4), BE(5)}. This keeps all 7 rooms connected without cycles.

Cost change: New total = $2 + 2 + 3 + 4 + 4 + 5 = 20$. The cost increases by 4 compared with the original MST cost of 16.

Why: The removed route EF was the cheapest link into the E-F-G cluster. Replacing it requires the next cheapest connection involving E, which is BE(5).

Final Submission Checklist

QUALITY CHECK

This document contains one fresh case study from each of CO1, CO2, and CO3. Each case study follows a 15-mark pattern with a scenario, diagram, sub-questions, and complete solution key.

- CO mapping included.
- BTL 4 included.
- Diagrams included for AVL tree, segment tree, and graph/MST.
- Solutions include construction, algorithm trace, code/pseudocode, and complexity/justification.